

PROJET DE GÉNIE LOGICIEL II

Guide de réalisation : cahier des charges, conception, développement, dépôt GitHub et présentation

Année académique : 2025-2026

Dates clés : CDC avant le 30 décembre 2025 | Application finale avant le 4 janvier 2026 | Présentation le 5 janvier 2026

Auteur : Bayang Souloukna

1. Objectif du projet

Ce projet vise à vous faire suivre une démarche complète de génie logiciel, de la spécification des besoins jusqu'à la livraison d'une application fonctionnelle. Vous travaillez en groupe, avec une évaluation à la fois collective (qualité globale) et individuelle (implication et maîtrise personnelle). La réussite se mesure autant par la qualité du produit que par la qualité du processus (organisation, documentation, tests et collaboration).

2. Livrables, calendrier et règles de dépôt

Tous les livrables doivent être versionnés et déposés sur GitHub. Le dépôt doit être accessible à l'enseignant (public ou privé selon consigne).

2.1 Dates et contenu attendus

Livrable	Contenu minimum	Date limite	Dépôt
Livrable 1 : Cahier des charges (CDC)	Document CDC (fonctionnel + technique) + annexes (maquettes, diagrammes si disponibles)	Mardi 30 décembre 2025 (23h59)	GitHub : /docs/cdc
Livrable 2 : Application finale	Code source + tests + procédure d'installation + guide d'utilisation + rapport de lancement	Dimanche 4 janvier 2026 (23h59)	GitHub : /src (ou /app) + /tests
Livrable 3 : Présentation finale	Slides + scénario de démo + répartition des prises de parole	Lundi 5 janvier 2026 (en classe)	GitHub : /docs/presentation (optionnel)

Règle importante : un dépôt GitHub sans instructions de lancement (README) sera pénalisé, même si le code est correct.

2.2 Travail en groupe et suivi individuel

L'évaluation comporte :

- une note de groupe (qualité globale des livrables et de la démonstration).
- une note individuelle (implication et maîtrise personnelle des éléments réalisés).

Votre implication individuelle sera mesurée à partir de traces objectives : commits, pull requests, issues, revues, commentaires et capacité à expliquer vos choix lors de la présentation.

2.3 Organisation recommandée du groupe

Définissez des rôles (adaptables) pour sécuriser l'avancement :

- Chef de projet : planification, suivi des jalons, cohérence des livrables.
- Responsable qualité : conventions, revues de code, tests, CI.
- Responsable conception : diagrammes, architecture, cohérence CDC -> conception.

- Développeurs (frontend/backend/mobile) : implémentation des fonctionnalités assignées.

Chaque rôle n'est pas exclusif : l'objectif est d'assurer qu'aucun aspect critique ne soit oublié.

3. Règles GitHub obligatoires

Pour standardiser vos dépôts, respectez les règles suivantes :

- Chaque fonctionnalité ou tâche doit être tracée via une Issue GitHub (titre clair, description, critères de fin).
- Le code doit être intégré via Pull Request (PR) avec une description de ce qui change et pourquoi.
- Les messages de commit doivent être courts et explicites (ex : 'Add login validation' au lieu de 'update').
- Utilisez des branches (feature/..., fix/...) et évitez de tout faire directement sur la branche main.
- Taguez les jalons : cdc-v1.0 (CDC) et v1.0.0 (application finale).

3.1 Structure minimale du dépôt

Arborescence recommandée :

```
/
docs/
  cdc/CDC.md
  conception/Architecture.md
  conception/Diagrammes/
  conception/Wireframes/
src/ (ou app/)
tests/
README.md
(optionnel) CONTRIBUTING.md
(optionnel) CHANGELOG.md
```

Le README doit au minimum contenir : objectif, membres, fonctionnalités, stack, installation, lancement et liens vers la documentation.

4. Cahier des charges : structure détaillée à respecter

Le cahier des charges (CDC) est le document de référence qui formalise le besoin et les exigences. Il doit être clair, testable, et suffisamment détaillé pour guider la conception et le développement.

4.1 Page de garde et informations générales

- Titre du projet, version, date, groupe (noms, matricules si applicable).
- Lien du dépôt GitHub.
- Résumé exécutif (5 à 10 lignes) : problème, solution proposée, utilisateurs cibles, valeur ajoutée.

4.2 Contexte, problématique et objectifs

À rédiger sous forme de paragraphes courts :

- Contexte : d'où vient le besoin ?
- Problématique : quel problème précis résolvez-vous ?
- Objectifs SMART : objectifs mesurables (réduction de temps, automatisation, fiabilité, traçabilité, etc.).

4.3 Périmètre : inclus / exclus

- In-scope : fonctionnalités incluses dans la version V1 (celles évaluées au 4 janvier 2026).
- Out-of-scope : fonctionnalités explicitement exclues (pour éviter les malentendus).
- Hypothèses et contraintes (connexion Internet, type d'appareil, langue, restrictions techniques, etc.).

4.4 Acteurs, rôles et droits

- Liste des acteurs (ex : utilisateur, administrateur, gestionnaire).
- Droits par acteur : créer, modifier, supprimer, consulter, valider, exporter...
- Règles d'accès : ce qui est autorisé/interdit selon le rôle.

4.5 Besoins fonctionnels (le cœur du CDC)

Les besoins fonctionnels doivent être exprimés sous forme de parcours, User Stories et exigences fonctionnelles (RF-xx). Chaque exigence doit être vérifiable.

4.5.1 Parcours utilisateurs (User Journeys)

- Décrivez 2 à 5 parcours complets, par exemple :
- Inscription -> Connexion -> Tableau de bord -> Action principale -> Confirmation.
- Recherche -> Filtrage -> Consultation -> Création -> Notification/Confirmation.

4.5.2 User Stories (format obligatoire)

- Pour chaque fonctionnalité, fournissez :
- Identifiant : US-01, US-02, ...
- Formulation : « En tant que [rôle], je veux [action] afin de [bénéfice] ».
- Priorité : Must / Should / Could.
- Critères d'acceptation : Given/When/Then (au moins 3).
- Cas limites : erreurs possibles, validations, exceptions, duplications.

Exemple (à adapter à votre sujet) :

US-03 : En tant qu'utilisateur, je veux réinitialiser mon mot de passe afin de récupérer l'accès à mon compte.
Priorité : Must

Critères d'acceptation :

- Étant donné un email existant, quand je demande une réinitialisation, alors je reçois un lien par email.
- Étant donné un lien expiré, quand je tente de changer le mot de passe, alors un message indique que le lien n'est plus valide.
- Étant donné un mot de passe non conforme, quand je valide, alors le système refuse et affiche les règles.

4.5.3 Exigences fonctionnelles (RF)

Listez vos exigences sous forme RF-xx. Chaque RF doit préciser : entrée, traitement, sortie et erreurs.

- RF-01 : Le système doit permettre à un utilisateur de créer un compte avec email unique.
- RF-02 : Le système doit permettre la recherche et le filtrage par critères.
- RF-03 : Le système doit journaliser les actions critiques (création, suppression, validation).

4.6 Données et règles métier

- Entités principales (ex : Utilisateur, Produit, Commande, Réservation...).
- Champs et contraintes (unicité, formats, champs obligatoires).
- Règles métier (ex : un statut « validé » ne peut pas revenir à « brouillon »).

4.7 Exigences non fonctionnelles (RNF)

Les RNF doivent être concrètes, idéalement chiffrées :

- Performance : ex. 95% des opérations répondent en moins de 2 secondes.
- Sécurité : hash des mots de passe, rôles, validation d'entrée, protections web si nécessaire.
- Disponibilité : gestion des erreurs, sauvegardes, reprise après incident.
- Qualité : tests automatisés, lint/format, conventions de code.
- UX : responsive, messages d'erreur compréhensibles, accessibilité de base.

4.8 Maquettes (UI/UX)

Fournir des maquettes (wireframes) pour au moins :

- Accueil / connexion.
- Écran principal (tableau de bord / liste).
- Création / édition.
- Détail (consultation).

4.9 Critères de recette (tests d'acceptation)

Construisez une liste de tests client. Format recommandé : Fonction -> Scénario -> Résultat attendu -> OK/KO.

4.10 Risques et plan de mitigation

Identifiez 3 à 6 risques et proposez une mitigation réaliste :

- Risque : retard (mitigation : découpage en lots + jalons intermédiaires).
- Risque : difficulté technique (mitigation : prototype tôt + choix de technologies maîtrisées).
- Risque : intégration (mitigation : PR fréquentes + CI + revues).

4.11 Planification

Présentez un planning simple, mais réaliste : lots/sprints, tâches, responsables et dates.

5. Conception : attendu et contenu

La conception traduit le CDC en solution technique cohérente. Elle doit expliciter l'architecture, les composants, les données, les interfaces, et justifier les choix.

5.1 Architecture globale

- Type d'architecture : MVC, client-serveur, Clean Architecture, etc.
- Découpage : frontend, backend, base de données, services externes.
- Contraintes de sécurité et de déploiement (si applicable).
- Justification des choix (2 à 5 lignes par décision importante).

5.2 Diagrammes minimum

Vous devez fournir au minimum :

- Modèle de données (ERD ou équivalent).
- Diagramme de classes ou modèle domaine (entités et relations).
- 1 à 2 diagrammes de séquence sur des scénarios clés.
- Diagramme de composants/modules (organisation du code).
-

5.3 Conception des API (si application web/mobile)

Pour chaque endpoint : méthode, route, authentification, body/params, validations, codes de retour et exemples de réponse.

5.4 Stratégie de tests

- Tests unitaires sur la logique métier (services, validateurs, calculs).
- Tests d'intégration sur l'API et la base de données (CRUD, sécurité, permissions).
- CI (intégration continue) : build + tests à chaque PR si possible.

6. Réalisation : exigences de qualité du code

Votre application finale doit respecter les points suivants :

- Installation simple : un lecteur externe doit pouvoir lancer le projet via les instructions du README.
- Structure claire : séparation des couches (présentation, logique, accès aux données).
- Gestion des erreurs : messages explicites, pas de crash non géré.
- Sécurité de base : validation des entrées, gestion des sessions/tokens selon le cas.
- Journalisation (logs) sur les actions importantes si pertinent.

6.1 Documentation minimale attendue

- README : installation, configuration, lancement, tests, et description rapide des fonctionnalités.
- Guide utilisateur (docs/guide-utilisateur.md ou section du README) : comment utiliser l'application.
- Guide développeur (optionnel) : architecture, conventions, scripts utiles.

7. Évaluation (groupe et individuelle)

7.1 Exemple de barème (indicatif)

Critère	Attendus	Points (sur 20)
Cahier des charges	Clarté, exhaustivité, exigences testables, maquettes, recette	5
Conception	Architecture + diagrammes + cohérence avec le CDC	4
Fonctionnalités livrées	Couverture du périmètre V1, stabilité, gestion des cas limites	4
Qualité du code	Structure, lisibilité, bonnes pratiques, modularité	3
Tests + CI	Tests automatisés pertinents, exécution simple	2
Documentation	README complet + guide utilisateur	1
Présentation	Démo, explications, réponses aux questions	1

7.2 Évaluation individuelle

L'enseignant pourra ajuster la note individuelle à partir de :

- Traces GitHub : commits, PR, issues, commentaires et revues.
- Qualité des contributions (pas seulement la quantité).
- Capacité à expliquer ses choix et sa partie du code pendant la présentation.
- Évaluation par les pairs (si mise en place).

8. Checklists de validation avant dépôt

8.1 Avant le 30 décembre 2025 (CDC)

- Le CDC est complet et suit la structure imposée.

- Chaque User Story a des critères d'acceptation testables.
- Les exigences non fonctionnelles sont concrètes (performance, sécurité, qualité).
- Les maquettes principales sont fournies.
- Le dépôt contient /docs/cdc/CDC.md et le tag cdc-v1.0.

8.2 Avant le 4 janvier 2026 (application finale)

- Le projet s'installe et se lance en suivant le README.
- Les fonctionnalités Must du CDC sont implémentées.
- Les tests s'exécutent (commande claire) et passent.
- La structure du dépôt est propre (src, tests, docs).
- Un tag v1.0.0 est créé et pointe vers la version stable.

8.3 Pour la présentation du 5 janvier 2026

- Scénario de démo prêt (parcours complet, données de test).
- Chaque membre connaît sa partie et peut répondre aux questions.
- Slides courtes : problème -> solution -> architecture -> démo -> bilan.
- Plan B : compte de test / jeu de données / capture d'écran en cas d'imprévu.